



Figure 2: Index page



Figure 3: Functions page

`malloc()` and `free()`. The `malloc` function allocates memory and returns a void pointer pointing to the allocated space. After the memory is freed, the variable pointing to that memory is called a dangling pointer. Using dangling pointers causes security vulnerabilities and the C compiler does not provide any method of avoiding them.

In contrast to C, the Rust language does not support dangling pointers. The allocation and initialisation of the memory is done at the same time using the allocation operator `'~'`, which returns a smart pointer. The compiler will automatically free the memory when the pointer goes out of scope. The programmer need not do it manually and hence avoids dangling pointers from being created.

Expressions

Rust is an expression based language. We can also say that the fundamental difference between the Rust syntax and languages like C is that some things that are statements in C are expressions in Rust. The command `'if'` is an expression whereas `'let'` is a statement in Rust.

In C, if we want to assign a value to a variable based on some condition, it is done in the following way:

```
int var;
if (value < 3) {
    var = 2;
}
else if (value > 5) {
    var = 6;
}
else {
    var = 4;
}
```

The same thing can be done in Rust, as follows:

```
let var =
```

```
if value < 3 {
    2
}
else if value > 5 {
    6
}
else {
    4
};
```

Here, the `'if'` condition is an expression; it returns some value, depending on the condition that holds. It is less error-prone and is more comfortable. But Rust also supports the common syntax and hence the coder can use either of the two ways.

Rust documentation

Rustdoc is a build in the API-documentation tool in Rust, which generates HTML documentation from *doc comments*. In order to generate documentation, it is necessary to include some documentation related lines inside the code.

Creating the documentation

When something has to be documented, we can use *doc comments*. The following is a sample set of lines that describes the documentation functionality written in *myworld.rs* file:

```
// This is not a doc comment
#![crate_id = "myworld"]
#![crate_type = "lib"]

///! This is my world for documentation. This is shown on the
crate index page. The
///! e ! sign indicates that the comment apply to the parent
of the comment, not to
///! what follows.

///! These doc comments will be shown in the Widget's
documentation.
pub struct Widgets{
    purpose: String,
    ///!These are not readable
    understandable: bool
}

///! These will be shown in the recalibrate part.
pub fn recalibrate(){
    ///! This is also a doc comment. This will show three
dots to show continuation.
    /* ... */
}
```

The HTML documentation for the above file can be